

NUMERICAL ANALYSIS

PROBLEM SET 3

ROB TOMPKINS

We begin by noting that all of the files referenced in this document are available at:
<https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/ProblemSet3>

1. PROBLEM 1

We'll be approximating $\sqrt{2}$ though this problem

1.a. Our goal is to find a polynomial interpolant $p(x)$ using the Lagrange form given that we have three points in the form $x, f(x)$ as $\{(-1, 1/2), (0, 1), (1, 2)\}$. So we let $f(x) = 2^x$ such that $f(1/2) = \sqrt{2}$. So our polynomial begins as

$$p(x) = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x),$$

with $y_i = f(x_i)$ and $L_i(x)$ being the i^{th} Lagrange polynomial. Computing the polynomials follows as

$$\begin{aligned} L_{2,0}(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - 0)(x - 1)}{(-1 - 0)(-1 - 1)} = \frac{x(x - 1)}{2} \\ L_{2,1}(x) &= \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x + 1)(x - 1)}{(0 + 1)(0 - 1)} = 1 - x^2 \\ L_{2,2}(x) &= \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{(x + 1)(x - 0)}{(1 + 1)(1 - 0)} = \frac{x(x + 1)}{2} \end{aligned}$$

1.b. Putting these together we have the following polynomial:

$$\begin{aligned} p(x) &= \frac{1}{2} \cdot \frac{x(x - 1)}{2} + 1 \cdot (1 - x^2) + 2 \cdot \frac{x(x + 1)}{2} \\ &= \frac{x^2}{4} - \frac{x}{4} + 1 - x^2 + x^2 + x \\ &= \frac{x^2}{4} + \frac{3x}{4} + 1 \end{aligned}$$

Pluggin in $x = 1/2$ we get the following

$$\begin{aligned} p\left(\frac{1}{2}\right) &= \frac{1}{4} \cdot \frac{1}{4} + \frac{3}{4} \cdot \frac{1}{2} + 1 \\ &= \frac{1}{16} + \frac{3}{8} + 1 \\ &= \frac{1}{16} + \frac{6}{16} + \frac{16}{16} \\ &= \frac{23}{16} \approx \sqrt{2}. \end{aligned}$$

1.c. For the error bound we want to compute the following

$$|f(x) - p(x)| = \frac{f'''(\xi)}{3!}(x - x_0)(x - x_1)(x - x_2),$$

where $\xi \in [-1, 1]$. We compute the third derivative of $f(x)$ by setting $f(x) = \ln(x) = e^{x \ln(2)}$. Clearly then we have

$$f'''(x) = (\ln(2))^3 e^{x \ln(2)} = (\ln(2))^3 2^x.$$

We want to have ξ such that $|f'''(\xi)|$ is largest so that everything else is beneath that as our upper bound. Because $f(x) = 2^x$ is an increasing exponential, the maximum of the third derivative will necessarily be the right hand end point. So we set $\xi = 1$ giving us $f'''(\xi) = 2(\ln(2))^3$. So

$$|f(x) - p(x)| = \frac{f'''(\xi)}{3!}(x - x_0)(x - x_1)(x - x_2) = \frac{2(\ln(2))^3}{6}(x + 1)(x)(x - 1)$$

And we want to know the error at $x = 1/2$ which comes out at

$$\begin{aligned} \left| f\left(\frac{1}{2}\right) - p\left(\frac{1}{2}\right) \right| &= \frac{2(\ln(2))^3}{6} \left(\frac{1}{2} + 1 \right) \cdot \frac{1}{2} \cdot \left(\frac{1}{2} - 1 \right) \\ &= \frac{3 \ln(2)^3}{24} \\ &= \frac{\ln(2)^3}{8} = 0.0416 \end{aligned}$$

2. PROBLEM 2

Our goal here is to approximate $\ln(2)$ with piecewise polynomials.

2.a. Since each piece has 3 coefficients and there are $n + 1$ points there are n pieces to the polynomial. So we have $3n$ coefficients to account for. We will refer to our interpolation function at $s(x)$, with the piceces being $s_i(x)$.

We begin by interpolating the endpoints of the segments giving us $s_i(x_i) = y_i$ and $s_i(x_{i+1}) = y_{i+1}$. This gives us our first $2n$ constraints.

Next we approach the first derivatives of the pieces for which we need equality for continuity on the inner $n - 1$ points or knots. So we must have that

$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}),$$

which gives us another $n - 1$ conditions. So we are missing 1 condition to reach our $3n$ requisite conditions to be able to solve for

The piece that we are missing is one of the boundary conditions. For example we could use $s_0(x_0)$ and fix that or use $s_{n-1}(x_n)$. Regardless we need this one more condition to get all of our constants.

2.b. We were instructed to use the dataset from problem 1, $\{(-1, 1/2), (0, 1), (1, 2)\}$ to solve this problem. This yields the intervals $I_0 = [-1, 0]$ and $I_1 = [0, 1]$. The interpolating polynomials that we are looking to find the coefficients for follow

$$\begin{aligned} s_0(x) &= a_0 + b_0(x + 1) + c_0(x + 1)^2 \\ s_1(x) &= a_1 + b_1x + c_1x^2. \end{aligned}$$

We begin by using the interpolation conditions such that $s_0(-1) = 1/2 \Rightarrow a_0 = 1/2$, $s_0(0) = 1 = 1/2 + b_0 + c_0$ and $s'_0(-1) = b_0 + 2c_0(-1 + 1) = 0$ we have that $b_0 = 0$. Thus $c_0 = 1/2$, which gives us

$$s_0(x) = \frac{1}{2} + \frac{1}{2}(x + 1)^2$$

Notice that $s_1(0) = 1 \Rightarrow a_1 = 1$. Next we use derivative continuity at $x = 0$ to find the other coefficients. We have that $s'_1(x) = b_1 + 2c_1x$, so $s'_0(0) = s'_1(0) = b_1 = 1$. Lastly, we use our boundary condition $s_1(1) = 2 = 1 + 1 + c_1(1) = 2$ to get $c_1 = 0$. And we have

$$s_1(x) = 1 + x.$$

So

$$s(x) = \begin{cases} \frac{1}{2} + \frac{1}{2}(x + 1)^2 & -1 \leq x < 0 \\ 1 + x & 0 \leq x \leq 1 \end{cases}$$

2.c. We begin by noticing that $s'_0(0) = 1$ and $s'_1(0) = 1$ by the continuity of $s'(x)$. Now again we are using the function $f(x) = 2^x$ such that $f'(x) = \ln(2)2^x$ and $f'(0) = \ln(2)$, so we use $s'(0) = 1 \approx \ln(2)$. We did a calculation on a computer just to test the error here, and we got $\ln(2) = 0.69315$, so we are quite off

2.d. Skip

3. PROBLEM 3

We're working with the data points $\{(0, 2), (1, -1), (2, 4)\}$

3.a. We begin by approaching Neville's algorithm around $\bar{x} = 1.3$ by hand in the following fashion

$$\begin{aligned} x_0 = 0, & & P_0(1.3) = 2, \\ x_1 = 1, & & P_1(1.3) = -1, & & P_{0,1}(1.3) = -1.9, \\ x_2 = 2, & & P_2(1.3) = 4, & & P_{1,2}(1.3) = 0.5, & & P_{0,1,2}(1.3) = 0.34. \end{aligned}$$

These values come from the following Lagrange Polynomials:

$$\begin{aligned} P_{0,1}(1.3) &= \frac{(1.3 - x_0)P_1(1.3) - (1.3 - x_2)P_0(1.3)}{x_2 - x_0} \\ &= \frac{(1.3 - 0)(-1) - (1.3 - 1)(2)}{1 - 0} \\ &= \frac{-1.3 - 0.6}{2} = -1.9 \end{aligned}$$

and

$$\begin{aligned} P_{1,2}(1.3) &= \frac{(1.3 - x_1)P_2(1.3) - (1.3 - x_2)P_0(1.3)}{x_2 - x_1} \\ &= \frac{(1.3 - 1)(4) - (1.3 - 2)(-1)}{2 - 1} \\ &= \frac{1.2 - 0.7}{1} = 0.5 \end{aligned}$$

and

$$\begin{aligned} P_{0,1,2}(1.3) &= \frac{(1.3 - x_0)P_1(1.3) - (1.3 - x_2)P_0(1.3)}{x_2 - x_0} \\ &= \frac{(1.3 - 0)(0.5) - (1.3 - 2)(-1.9)}{2 - 0} \\ &= \frac{0.65 - 1.33}{2} = -0.34 \approx f(1.3). \end{aligned}$$

We also have Matlab code for running this approximation using Neville's algorithm given as

```
function p = nevilleInterpolation(x, y, xbar)
    n = length(x);
    Q = zeros(n, n);

    % First column
    for i = 1:n
        Q(i,1) = y(i);
    end

    % Build Neville table
    for j = 2:n
        for i = 1:n-j+1
            Q(i,j) = ((xbar - x(i)) * Q(i+1,j-1) - (xbar - x(i+j-1)) * Q(i,j-1)) / (x(i+j-1) - x(i));
        end
    end
```

```

end

p = Q(1,n);

% Optional: display the table
disp('Neville table:');
disp(Q);
end

x = [0 1 2];
y = [2 -1 4];
xbar=1.3;

ybar = nevilleInterpolation(x,y,xbar)

```

And the algorithm yields $\mathbf{xbar} = -0.3400$.

3.b. We want to compute the same as above except use Newton's method using divided differences. So we need to find the polynomial:

$$P_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1),$$

where $f[x_i]$ is the divided difference with respect to x_i , and

$$f[x_i, x_{i+1}, \dots, x_{i+k-1}, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k-1}, x_{i+k}] - f[x_i, x_{i+1}, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

We begin by noticing that $f[x_i] = f(x_i)$ is given by our points: $x_0 = 0$, $x_1 = 1$, and $x_2 = 2$, so

$$f[x_0] = 2, \quad f[x_1] = -1, \quad f[x_2] = 4.$$

Now we compute $f[x_0, x_1]$ and $f[x_1, x_2]$ as

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{-1 - 2}{1} = -3$$

and

$$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1} = \frac{4 + 1}{2 - 1} = 5.$$

Lastly $f[x_0, x_1, x_2]$ follows as:

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{5 + 3}{2} = 4$$

So our polynomial follows as:

$$\begin{aligned} P_2(x) &= 2 - 3(x - 0) + 4(x - 0)(x - 1) \\ &= 4x^2 - 7x + 2 \end{aligned}$$

So when we compute $P_2(1.3)$ we get the following

$$\begin{aligned} P_2(1.3) &= 4(1.69) - 7(1.3) + 2 \\ &= 6.76 - 9.1 + 2 \\ &= -0.34 \end{aligned}$$

which is what we got in 3.a using Neville's method. Similar to the previous problem we code Newton's method as:

```
function p = newtonDividedDiff(x, y, xbar)
    n = length(x);
    DD = zeros(n, n);

    % First column: function values
    DD(:,1) = y(:);

    % Build divided difference table
    for j = 2:n
        for i = 1:n-j+1
            DD(i,j) = (DD(i+1,j-1) - DD(i,j-1)) / (x(i+j-1) - x(i));
        end
    end

    % Evaluate Newton polynomial at xbar
    p = DD(1,1);
    prodterm = 1;

    for j = 2:n
        prodterm = prodterm * (xbar - x(j-1));
        p = p + DD(1,j) * prodterm;
    end

    % Display divided difference table if desired
    disp('Divided difference table:')
    disp(DD)
end

x = [0 1 2];
y = [2 -1 4];
xbar=1.3;

ybarNewton = newtonDividedDiff(x, y, xbar)
```

which again yields $\text{xbar} = -0.3400$.